Nombre: Hernández Silva Roberto Jaziel	**Matrícula**: 1918436
Nombre: Martínez Álvarez Alejandro	**Matrícula**: 2052641
Nombre: Rodríguez Ortiz Danna Saraí	**Matrícula**: 2060308
Nombre del curso: Componentes de Lógica de Negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 4	**Actividad**: Laboratorio guiado. Programa para administrar pedidos de una tienda ficticia TechStore
Fecha: 28/10/2026	

Bibliografia (Formato APA)

- Evans, E. (2003). Domain-driven design: Tackling complexity in the heart of software. Addison-Wesley.
- Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.
- Facultad de Ingeniería Mecánica y Eléctrica, Universidad Autónoma de Nuevo León. (s. f.). Desarrollo de componentes de negocio: Manual de lectura ilustrado (Unidad 4) [Material de clase].

Objetivo:

Implementar los componentes de lógica de negocio de TechStore mediante el uso de entidades DDD, Specification Pattern, máquinas de estados, Domain Events y una API REST, aplicando las reglas de los pedidos y realizando pruebas para comprobar su correcto funcionamiento.

1. Procedimiento y desarrollo

La práctica se trabajó con los datos de la tienda ficticia TechStore. Primero se cargaron los archivos CSV; después se revisaron las clases del pedido y las reglas de descuento. Finalmente se documentaron los cambios de estado, los eventos y las respuestas de la API. El sistema utiliza objetos en memoria, por lo que al iniciar nuevamente el programa los pedidos vuelven a su estado inicial.

1.1. Archivos utilizados

Archivo o carpeta	Función en la práctica
datos/	Contiene los CSV de clientes, productos, pedidos, detalles y escenarios.
cargar_csv.py	Lee la información y crea los objetos utilizados por el programa.
app/domain.py	Define las entidades, los estados, las validaciones y los eventos del pedido.
app/specifications.py	Contiene las condiciones empleadas para evaluar descuentos.
app/api.py	Permite utilizar las operaciones de TechStore desde peticiones HTTP.
tests/	Contiene las pruebas automatizadas que se ejecutan con pytest.

Los CSV del paquete incluyen 12 clientes, 12 productos, 16 pedidos, 21 renglones de detalle y 15 escenarios (E01–E15). Cada pedido comienza en DRAFT.

1.2. Entidades y Value Objects

Clase	Uso principal	Regla que debe cuidar
Money	Manejar importes en MXN mediante Decimal.	No aceptar cantidades negativas ni operaciones entre monedas diferentes.
Customer	Representar al cliente y sus características.	Identificar si es VIP y si está activo.
OrderItem	Guardar producto, cantidad y precio unitario.	La cantidad debe ser positiva y el subtotal se calcula, no se captura.
Order	Representar un pedido con identificador propio.	Controlar artículos, límite de 20 unidades y cambios de estado mediante métodos.

OrderStatus	Definir los seis estados del pedido.	Evitar estados que no estén contemplados.
DomainEvent	Registrar una operación de negocio ya realizada.	Guardar el evento únicamente después de un cambio válido.

1.3. Reglas que se comprueban

En TechStore el límite es de 20 unidades por pedido, no de 20 renglones. Además, los artículos solo pueden agregarse mientras el pedido está en DRAFT. Para confirmar, el pedido necesita al menos un artículo y un cliente activo. En los siguientes apartados se muestra cómo estas condiciones afectan los descuentos y las transiciones.

2. Specification Pattern y descuentos

Las Specifications permiten evaluar condiciones sin cambiar el pedido. En esta práctica se utilizan para distinguir clientes VIP, revisar el total, comprobar el número de unidades y detectar promociones activas. También pueden combinarse con AND, OR y NOT.

Condición	Qué revisa	Ejemplo del paquete
MinimumAmountSpec(500)	Que el importe sea mayor a 500 MXN, no igual.	O010 tiene exactamente 500.00 MXN.
VIPCustomerSpec()	Que el cliente tenga la condición de VIP.	O001 pertenece a un cliente VIP.
MinimumUnitsSpec(4)	Que el pedido tenga al menos cuatro unidades.	O011 tiene solo tres unidades.
ActivePromotionSpec()	Que el pedido tenga una promoción activa.	O005 tiene promoción y queda excluido.

2.1. Reglas de descuento

Descuento	Condiciones
10 %	Cliente VIP AND total > 500 MXN AND unidades ≥ 4 AND NOT promoción.
5 %	Total > 5000 MXN AND NOT promoción.
0 %	Si no se cumple alguna de las dos reglas anteriores.

Cuando se cumplen las dos alternativas, la regla del 10 % tiene prioridad. El descuento se calcula a partir del total original; evaluar la regla no modifica los artículos ni el estado del pedido.

2.2. Ejemplo del pedido O001

O001 tiene dos renglones: SKU001 × 2 a $600.00 y SKU002 × 2 a $450.00. Esto representa 4 unidades y un importe total de $2,100.00 MXN. Como el cliente es VIP, no tiene promoción activa y supera $500.00, le corresponde el descuento del 10 %.

Cálculo	Resultado
Total original	$2,100.00 MXN
Descuento (10 %)	$210.00 MXN
Total con descuento	$1,890.00 MXN

Otro ejemplo es O006, que alcanza $18,000.00 MXN y no recibe el descuento VIP. Al superar $5,000.00 y no tener promoción activa, corresponde aplicar el 5 %, equivalente a $900.00 MXN.

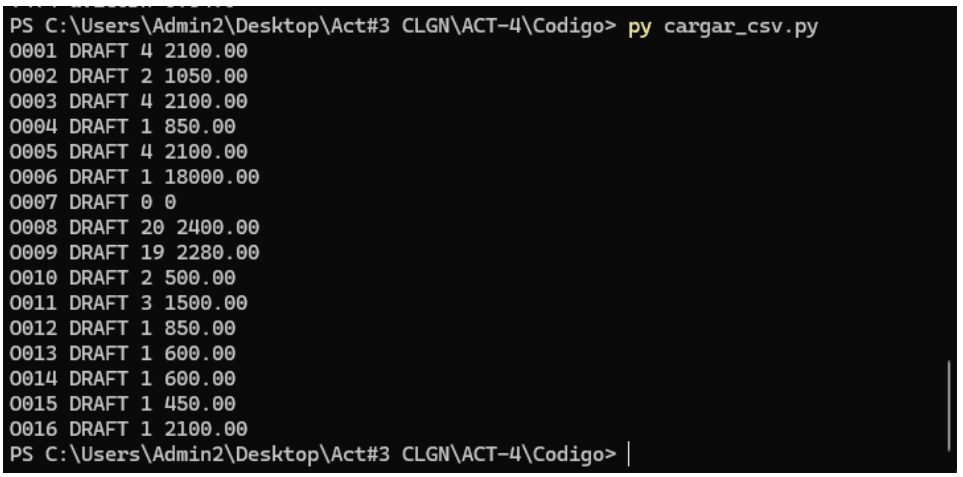
2.3. Dos valores que requieren atención

E07 utiliza el pedido O008, con exactamente 20 unidades. Eso está permitido, así que debe poder confirmarse si se cumplen las demás condiciones. En cambio, E08 intenta agregar dos unidades a un pedido que ya tiene 19; como la suma sería 21, la operación debe rechazarse sin agregar el artículo.

En E09, O010 tiene un total de exactamente $500.00, pero también es un cliente no VIP y tiene dos unidades. Por sí solo, este caso no comprueba de manera aislada el límite estricto de $500.00; para ello se necesitaría una prueba adicional con las demás condiciones cumplidas.

3. Carga los datos

Captura de la terminal con los 16 pedidos cargados.



4. Ejecuta los 15 escenarios

Caso	Estado inicial	Operación	Resultado real	Explicación
E01	DRAFT	Aplicar regla 10%	Descuento aplicado	Cumple VIP, >500 MXN, 4 unidades y sin promoción.
E02	DRAFT	Evaluar regla de descuento	Ningún descuento aplicado	No cumple VIP, sin promoción y <5000 MXN.
E03	DRAFT	Confirmar pedido (confirm())	Lanza InvalidOrderData (Rechazado)	El cliente VIP está inactivo (active = False).
E04	DRAFT	Evaluar regla de descuento	Ningún descuento aplicado (0%)	La promoción activa (has_promotion = True) excluye los descuentos.
E05	DRAFT	Evaluar regla de descuento	Descuento del 5% aplicado	Sin promoción, cliente no VIP, pero total > 5000 MXN.
E06	DRAFT	Confirmar pedido (confirm())	Lanza InvalidOrderData (Rechazado)	El pedido no contiene artículos (pedido vacío).
E07	DRAFT	No confirmar.	No se confirma la orden	La orden no contiene artículos.
E08	DRAFT	add_item() con 2 unidades de SKU003	Falla (OrderItemLimitExceeded)	El pedido O009 ya contaba con 19 unidades. Intentar sumar 2 superaría el límite permitido de 20 unidades.
E09	DRAFT	Evaluar regla de descuento	0% (Sin descuento)	El total del pedido O010 es exactamente $500.00 MXN. La especificación MinimumAmountSpec exige que sea estrictamente mayor a 500 (> 500).

Caso	Estado inicial	Operación	Resultado real	Explicación
E10	DRAFT	Evaluar regla de descuento	0% (Sin descuento)	Aunque el cliente es VIP, el pedido O011 solo tiene 3 unidades y MinimumUnitsSpec exige al menos 4 unidades (>= 4).
E11	DRAFT	confirm() y luego ship("")	Falla en envío (InvalidOrderData)	El método confirm() pasa a CONFIRMED emitiendo OrderConfirmed, pero ship() falla y permanece en CONFIRMED al recibir una guía vacía.
E12	DRAFT	Ejecutar confirm() dos veces consecutivas	Falla en 2º confirmación (InvalidOrderTransition)	La primera confirmación pasa a CONFIRMED emitiendo OrderConfirmed. La segunda falla por no estar en DRAFT y no emite ningún nuevo evento.
E13	DRAFT	confirm(), ship("GUIA123") y cancel("motivo")	Falla en cancelación (InvalidOrderTransition)	El pedido cambia de DRAFT → CONFIRMED → SHIPPED. La cancelación falla porque solo se permite cancelar pedidos en estado CONFIRMED.
E14	DRAFT	confirm(), ship("GUIA123") y return_order()	Éxito (Estado final: RETURNED)	Transita válidamente por DRAFT → CONFIRMED → SHIPPED → RETURNED, emitiendo los eventos OrderConfirmed, OrderShipped y OrderReturned.

Caso	Estado inicial	Operación	Resultado real	Explicación
E15	DRAFT	confirm(), ship("GUIA123") y deliver()	Éxito (Estado final: DELIVERED)	Transita válidamente por DRAFT → CONFIRMED → SHIPPED → DELIVERED, emitiendo los eventos OrderConfirmed, OrderShipped y OrderDelivered.

5. IMÁGENES DE LOS CASOS

Orden O001

```
PS C:\Users\Admin2\Desktop\Act#3 CLGN\ACT-4\Codigo> python
Python 3.15.2 (tags/v3.15.2:5e58eaa, Oct 25 2026, 17:27:03) [MSC v.1951 64 bi
t (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> from cargar_csv import load_fixture
>>> orders = load_fixture()
>>> order = orders["O001"]
>>> print(order.status, order.units, order.total.amount)
OrderStatus.DRAFT 4 2100.00
>>> order.confirm()
>>> print(order.status, [e.name for e in order.events])
OrderStatus.CONFIRMED ['OrderConfirmed']
>>> try:
...     order.confirm()
... except Exception as e:
...     print("Excepción capturada correctamente:", type(e).__name__, "-", e\
)
...
Excepción capturada correctamente: InvalidOrderTransition - Solo DRAFT
>>> print(order.status, order.units, order.total.amount)
OrderStatus.CONFIRMED 4 2100.00
>>>
```

Orden O002

```
>>> order = orders["O002"]
>>> print(order.status, order.units, order.total.amount)
OrderStatus.DRAFT 2 1050.00
>>> order.confirm()
>>>  print(order.status, [e.name for e in order.events])
  File "<python-input-11>", line 1
    print(order.status, [e.name for e in order.events])
IndentationError: unexpected indent
>>> print(order.status, [e.name for e in order.events])
OrderStatus.CONFIRMED ['OrderConfirmed']
>>>
```

Orden O003

```
>>> order = orders["O003"]
>>> print(order.status, order.units, order.total.amount)
OrderStatus.DRAFT 4 2100.00
>>> order.confirm()
Traceback (most recent call last):
  File "<python-input-15>", line 1, in <module>
    order.confirm()
    ~~~~~~~~~~~~~~^^
  File "C:\Users\Admin2\Desktop\Act#3 CLGN\ACT-4\Codigo\app\domain.py", line
237, in confirm
    raise InvalidOrderData("Requiere artículos y cliente activo")
app.domain.InvalidOrderData: Requiere artículos y cliente activo
>>>
```

Orden O004

```
>>> order = orders["O004"]
>>> print(order.status, order.units, order.total.amount)
OrderStatus.DRAFT 1 850.00
>>> order.confirm()
Traceback (most recent call last):
  File "<python-input-20>", line 1, in <module>
    order.confirm()
    ~~~~~~~~~~~~~~^^
  File "C:\Users\Admin2\Desktop\Act#3 CLGN\ACT-4\Codigo\app\domain.py", line 237, in confirm
    raise InvalidOrderData("Requiere artículos y cliente activo")
app.domain.InvalidOrderData: Requiere artículos y cliente activo
>>> try:
...     orders["O004"].confirm()
... except Exception as exc:
...     print(f"Bloqueado correctamente: {type(exc).__name__} -> {exc}")
...
Bloqueado correctamente: InvalidOrderData -> Requiere artículos y cliente activo
>>>
```

Orden O005

```
>>> order = orders["O005"]
>>> print(order.status, order.units, order.total.amount)
OrderStatus.DRAFT 4 2100.00
>>> try:
...     orders["O005"].confirm()
... except Exception as exc:
...     print(f"Bloqueado correctamente: {type(exc).__name__} -> {exc}")
...
>>>
>>> try:
...     orders["O005"].confirm()
... except Exception as exc:
...     print(f"Bloqueado correctamente: {type(exc).__name__} -> {exc}")
...
Bloqueado correctamente: InvalidOrderTransition -> Solo DRAFT
```

Orden O006

```
>>> order = orders["O006"]
>>> print(order.status, order.units, order.total.amount)
OrderStatus.DRAFT 1 18000.00
>>> order.confirm()
>>> print(order.status, [e.name for e in order.events])
OrderStatus.CONFIRMED ['OrderConfirmed']
>>> try:
...     orders["O006"].confirm()
...     except Exception as exc:
...     print(f"Bloqueado correctamente: {type(exc).__name__} -> {exc}")
...
  File "<python-input-32>", line 3
    except Exception as exc:
    ^^^^^^
SyntaxError: invalid syntax
>>> try:
...     orders["O006"].confirm()
... except Exception as exc:
...     print(f"Bloqueado correctamente: {type(exc).__name__} -> {exc}")
...
Bloqueado correctamente: InvalidOrderTransition -> Solo DRAFT
```

Orden O007

```
>>> order = orders["O007"]
>>> print(order.status, order.units, order.total.amount)
OrderStatus.DRAFT 0 0
>>>  order.confirm()
  File "<python-input-36>", line 1
    order.confirm()
IndentationError: unexpected indent
>>> order.confirm()
Traceback (most recent call last):
  File "<python-input-37>", line 1, in <module>
    order.confirm()
    ~~~~~~~~~~~~~^^
  File "C:\Users\Admin2\Desktop\Act#3 CLGN\ACT-4\Codigo\app\domain.py", line 237, in confirm
    raise InvalidOrderData("Requiere artículos y cliente activo")
app.domain.InvalidOrderData: Requiere artículos y cliente activo
>>> try:
...     orders["O007"].confirm()
... except Exception as exc:
...     print(f"Bloqueado correctamente: {type(exc).__name__} -> {exc}")
...
Bloqueado correctamente: InvalidOrderData -> Requiere artículos y cliente activo
>>>
```

Orden O008

```
>>> order = orders["O008"]
>>> print(order.status, order.units, order.total.amount)
OrderStatus.DRAFT 20 2400.00
>>> order.confirm()
>>> print(order.status, [e.name for e in order.events])
OrderStatus.CONFIRMED ['OrderConfirmed']
>>> try:
...     orders["O008"].confirm()
... except Exception as exc:
...     print(f"Bloqueado correctamente: {type(exc).__name__} -> {exc}")
...
Bloqueado correctamente: InvalidOrderTransition -> Solo DRAFT
>>>
```

Orden O009

```
>>> order = orders["O009"]
>>> print(order.status, order.units, order.total.amount)
OrderStatus.DRAFT 19 2280.00
>>> order.confirm()
>>> print(order.status, [e.name for e in order.events])
OrderStatus.CONFIRMED ['OrderConfirmed']
>>> try:
...     orders["O009"].confirm()
... except Exception as exc:
...     print(f"Bloqueado correctamente: {type(exc).__name__} -> {exc}")
...
Bloqueado correctamente: InvalidOrderTransition -> Solo DRAFT
>>>
```

Orden O010

```
>>> order = orders["O010"]
>>> print(order.status, order.units, order.total.amount)
OrderStatus.DRAFT 2 500.00
>>> order.confirm()
>>> print(order.status, [e.name for e in order.events])
OrderStatus.CONFIRMED ['OrderConfirmed']
>>> try:
...     orders["O010"].confirm()
... except Exception as exc:
...     print(f"Bloqueado correctamente: {type(exc).__name__} -> {exc}")
...
Bloqueado correctamente: InvalidOrderTransition -> Solo DRAFT
```

Orden O011

```
>>> order = orders["O011"]
... print(order.status, order.units, order.total.amount)
... order.confirm()
... print(order.status, [e.name for e in order.events])
...
OrderStatus.DRAFT 3 1500.00
OrderStatus.CONFIRMED ['OrderConfirmed']
>>> try:
...     orders["O011"].confirm()
... except Exception as exc:
...     print(f"Bloqueado correctamente: {type(exc).__name__} -> {exc}")
...
Bloqueado correctamente: InvalidOrderTransition -> Solo DRAFT
>>>
```

Orden O012

```
>>> order = orders["O012"]
... print(order.status, order.units, order.total.amount)
... order.confirm()
... print(order.status, [e.name for e in order.events])
...
OrderStatus.DRAFT 1 850.00
OrderStatus.CONFIRMED ['OrderConfirmed']
>>> try:
...     orders["O012"].confirm()
... except Exception as exc:
...     print(f"Bloqueado correctamente: {type(exc).__name__} -> {exc}")
...
Bloqueado correctamente: InvalidOrderTransition -> Solo DRAFT
>>>
```

Orden O013

```
>>> order = orders["O013"]
... print(order.status, order.units, order.total.amount)
... order.confirm()
... print(order.status, [e.name for e in order.events])
...
OrderStatus.DRAFT 1 600.00
OrderStatus.CONFIRMED ['OrderConfirmed']
>>> try:
...     orders["O013"].confirm()
... except Exception as exc:
...     print(f"Bloqueado correctamente: {type(exc).__name__} -> {exc}")
...
Bloqueado correctamente: InvalidOrderTransition -> Solo DRAFT
>>>
```

Orden O014

```
>>> order = orders["O014"]
... print(order.status, order.units, order.total.amount)
... order.confirm()
... print(order.status, [e.name for e in order.events])
...
OrderStatus.DRAFT 1 600.00
OrderStatus.CONFIRMED ['OrderConfirmed']
>>> try:
...     orders["O014"].confirm()
... except Exception as exc:
...     print(f"Bloqueado correctamente: {type(exc).__name__} -> {exc}")
...
Bloqueado correctamente: InvalidOrderTransition -> Solo DRAFT
>>>
```

Orden O015

```
>>> order = orders["O015"]
... print(order.status, order.units, order.total.amount)
... order.confirm()
... print(order.status, [e.name for e in order.events])
...
OrderStatus.DRAFT 1 450.00
OrderStatus.CONFIRMED ['OrderConfirmed']
>>> try:
...     orders["O015"].confirm()
... except Exception as exc:
...     print(f"Bloqueado correctamente: {type(exc).__name__} -> {exc}")
...
Bloqueado correctamente: InvalidOrderTransition -> Solo DRAFT
>>>
```

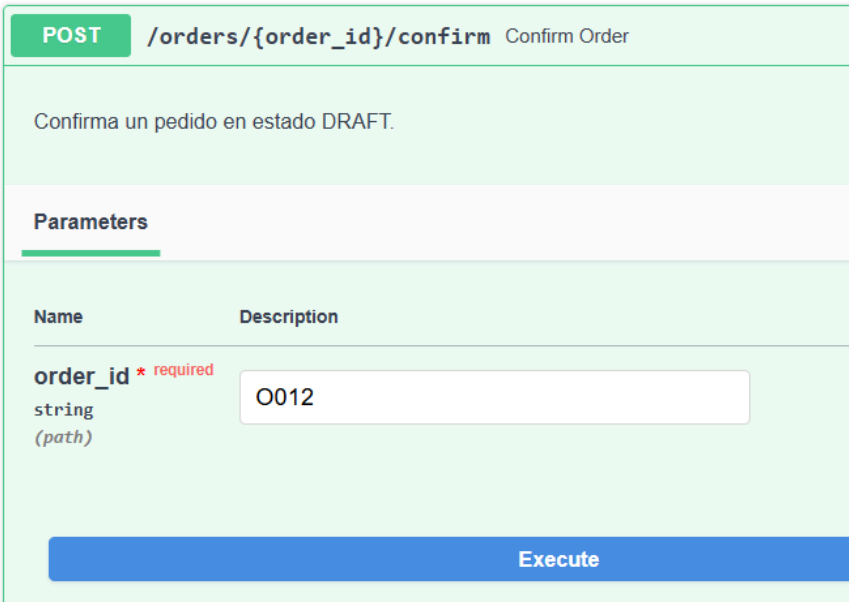
6. API REST y manejo de errores

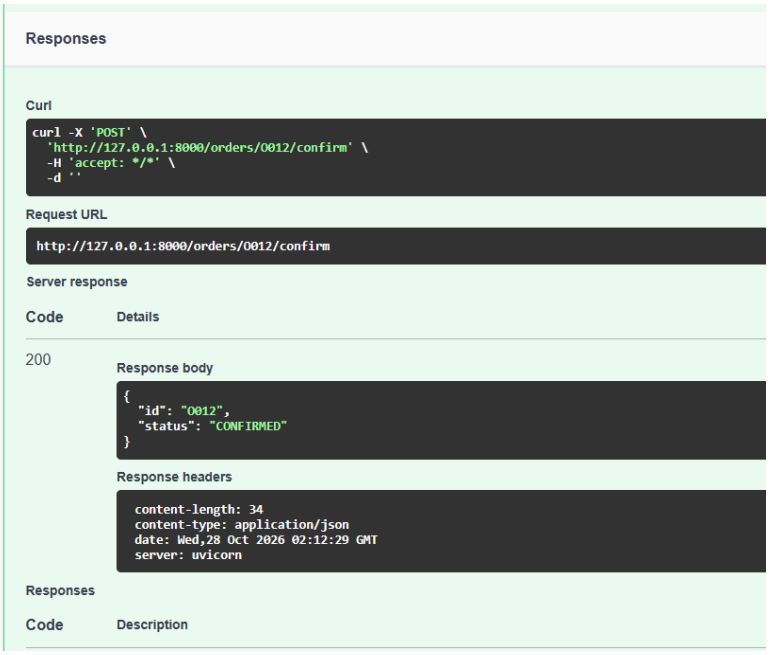
La API REST permite realizar operaciones sobre los pedidos desde Swagger. El reporte actual del equipo incluye una confirmación exitosa y ejemplos de tres respuestas de error. La API no debe repetir las reglas del dominio: su función es recibir la petición y devolver el resultado o el motivo del rechazo.

HTTP	Significado	Ejemplo en las capturas del equipo
200	La operación se realizó.	Confirmar O012 → CONFIRMED.
404	El pedido solicitado no existe.	Se intenta confirmar O019.
409	La operación no es válida para el estado actual.	Segunda confirmación de O013.
422	No se cumplen los datos o condiciones necesarias.	Confirmar O007 sin artículos.

7. SWAGGER

POST / Confirmar CODE 200

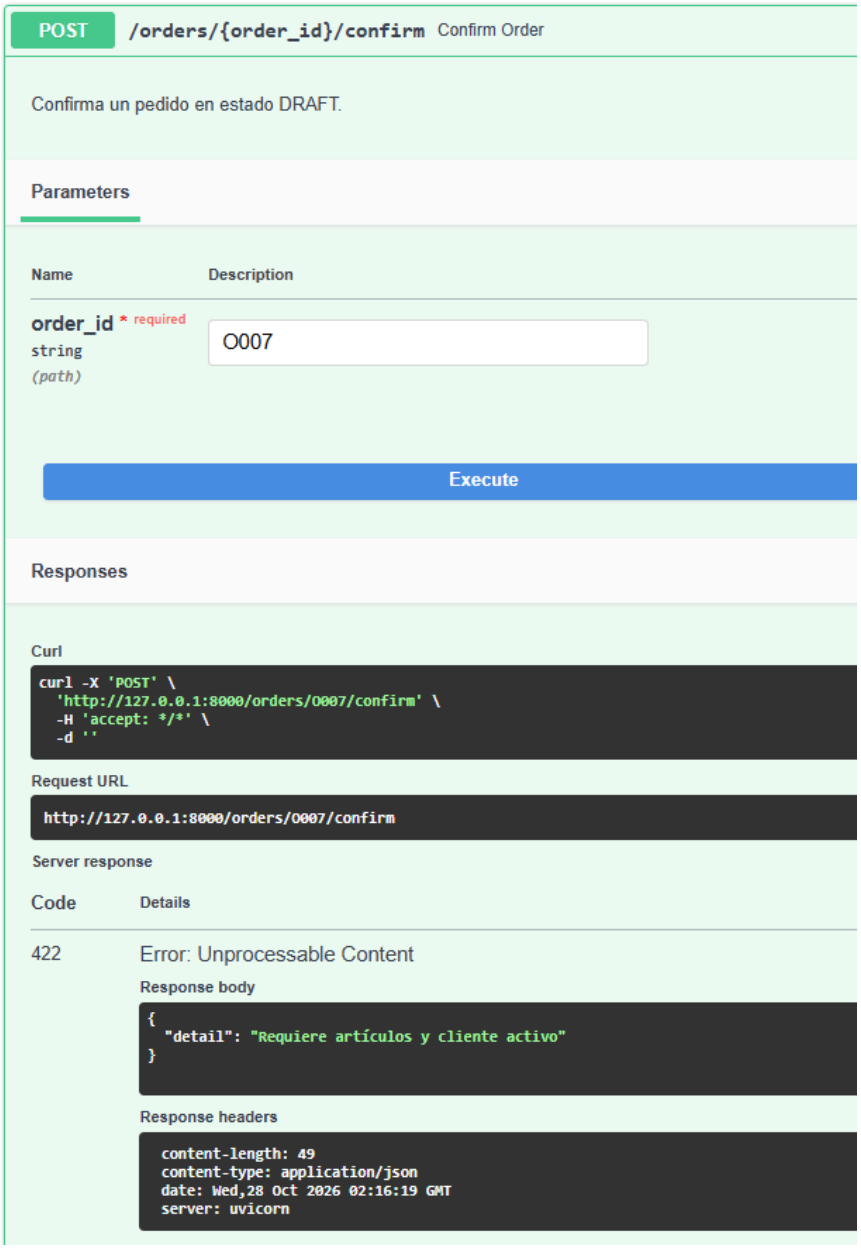




Código 200 (Éxito): Al enviar el ID de pedido O012, la operación se realiza correctamente y el pedido cambia su estado a "CONFIRMED"

Representa una respuesta exitosa, mostrando que el pedido se confirmó correctamente bajo el estado.

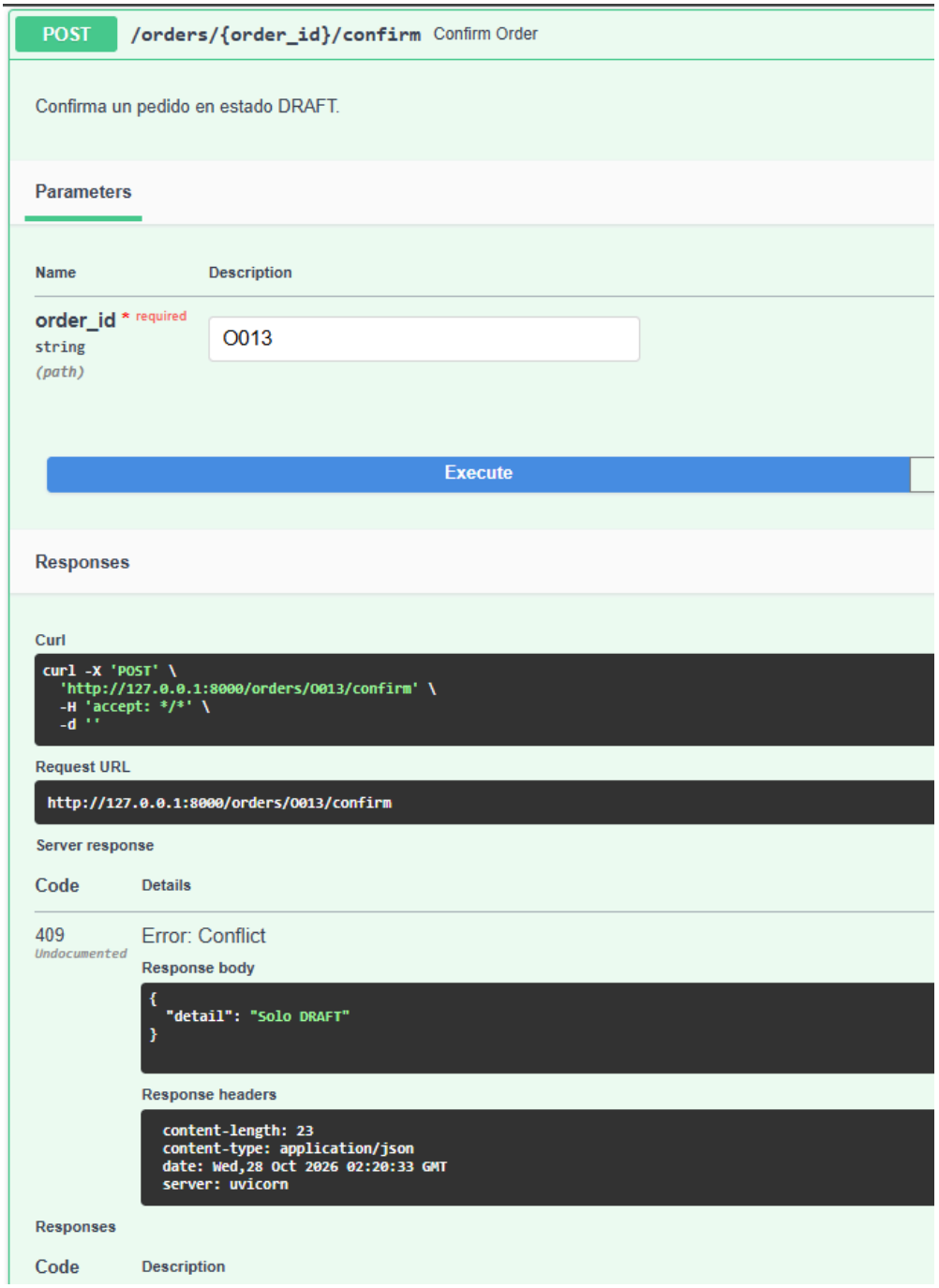
POST / Confirmar CODE 422



Código 422 (Unprocessable Content): Al enviar el ID O007, la solicitud no se puede procesar debido a reglas de negocio; el detalle indica "requiere artículos y cliente activo", lo que significa que le faltan elementos obligatorios al pedido en estado DRAFT para poder confirmarse.

Representa contenido no procesable (*Unprocessable Content*), indicando mediante el detalle que el pedido "requiere artículos y cliente activo" para poder avanzar.

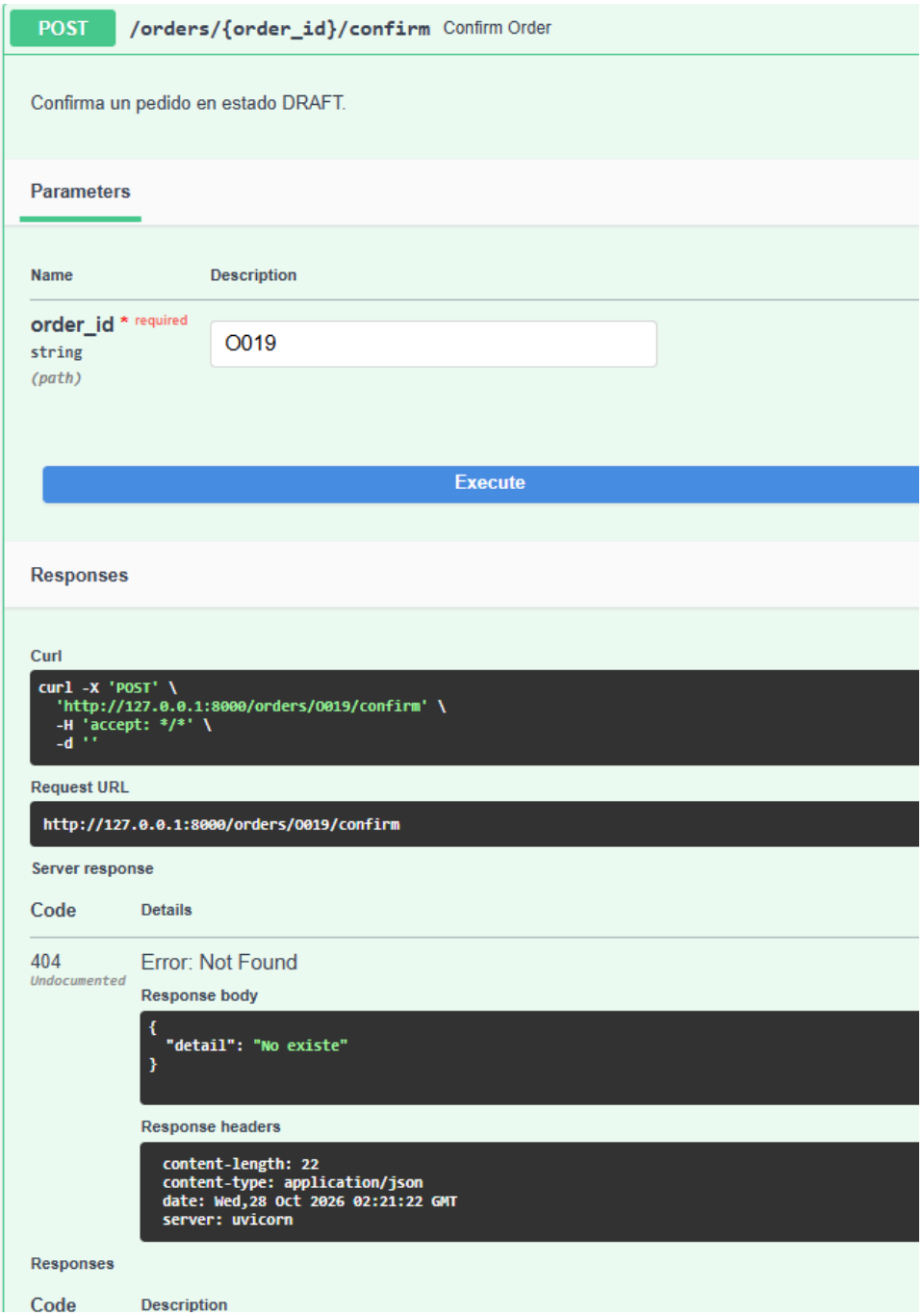
POST / Confirmar CODE 409



Código 409 (Conflict): Al enviar el ID O013, hay un conflicto con el estado actual ya que el detalle indica "solo DRAFT", lo que significa que se intentó confirmar un pedido que ya no se encuentra en ese estado.

Representa un conflicto (*Conflict*), indicando que la operación falló porque la regla exige que esté "solo DRAFT".

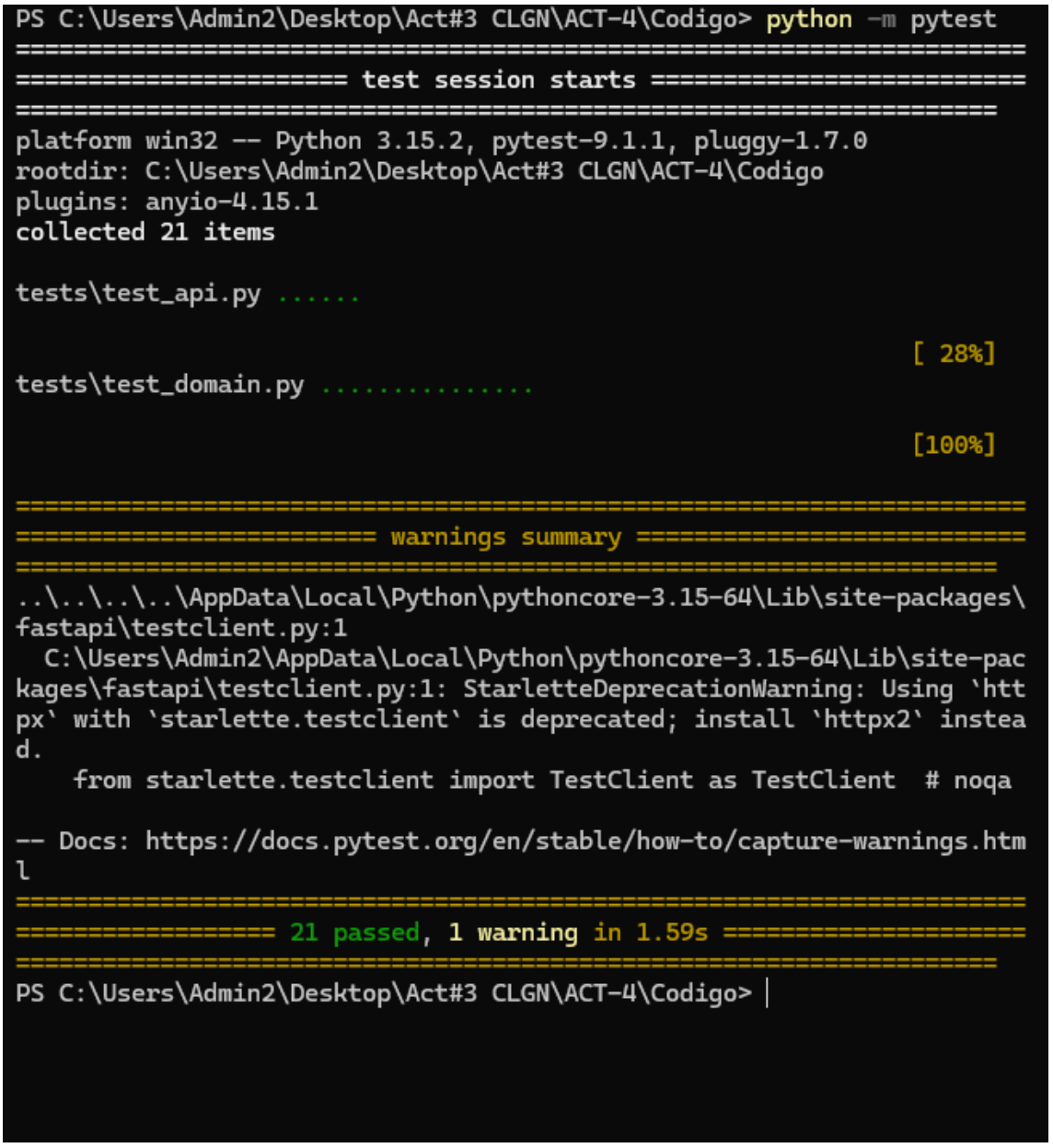
POST / Confirmar CODE 404



Código 404 (Not Found): Al enviar el ID O019, el servidor responde con un error 404 y el detalle "No existe", indicando que el identificador del pedido no se encuentra en el sistema.

Representa un recurso no encontrado (*Not Found*), indicando que el pedido consultado "No existe" en el sistema

8. PYTEST



Pruebas en Pytest: Se ejecutaron un total de 21 pruebas unitarias y de integración (21 passed), abarcando los módulos tests\test_api.py y tests\test_domain.py, en un tiempo total de 1.59 segundos.

Este resultado indica que absolutamente todas las pruebas pasaron de manera exitosa sin errores, validando que tanto la lógica de la API como la lógica del dominio de la aplicación funcionan correctamente tal como se espera.

9. Diagrama de estados y explicación de los Domain Events.



El pedido de TechStore comienza en el estado DRAFT y puede pasar por diferentes estados según las operaciones que se realicen. Cada transición debe cumplir las reglas del negocio.

Los estados finales son DELIVERED, RETURNED y CANCELLED. Una vez que el pedido llega a uno de ellos, no se permiten nuevas operaciones de transición.

10. Transiciones permitidas y sus condiciones

TRANSICIÓN	EVENTO GENERADO	CONDICIONES NECESARIAS
DRAFT → CONFIRMED	OrderConfirmed	El pedido debe estar en borrador, tener al menos un artículo y contar con un cliente activo.
CONFIRMED → SHIPPED	OrderShipped	El pedido debe estar confirmado y tener un número de guía válido, no vacío.

CONFIRMED → CANCELLED	OrderCancelled	El pedido debe estar confirmado y tener un motivo de cancelación no vacío.
SHIPPED → DELIVERED	OrderDelivered	El pedido debe estar enviado.
SHIPPED → RETURNED	OrderReturned	El pedido debe estar enviado.

11. ¿Qué son los Domain Events?

Los Domain Events o eventos de dominio representan hechos importantes que ya ocurrieron dentro del sistema. En TechStore, permiten registrar las acciones realizadas durante el ciclo de vida de un pedido.

Los cinco eventos son:

- OrderConfirmed: indica que el pedido quedó confirmado y que ya no se pueden agregar artículos.
- OrderShipped: indica que el pedido salió del almacén y tiene un número de guía.
- OrderCancelled: indica que el pedido fue cancelado por un motivo registrado.
- OrderDelivered: indica que el cliente recibió el pedido.
- OrderReturned: indica que el pedido enviado fue devuelto.

Cada evento contiene información como el nombre del evento, el identificador del pedido y la fecha y hora en que ocurrió.

12. ¿Por qué los eventos solo se emiten después de una transición válida?

Los eventos solamente deben generarse cuando se cumplen las condiciones necesarias para cambiar de estado. Esto permite mantener la consistencia de los datos y evitar acciones incorrectas.

El proceso tiene tres pasos:

1. **Validar las condiciones**: comprobar que el pedido se encuentre en el estado correcto y cumpla los requisitos.
2. **Cambiar el estado:** actualizar el estado del pedido si todas las validaciones fueron satisfactorias.
3. **Registrar el evento:** guardar el evento que corresponde a la transición realizada.

Por ejemplo, si un pedido está en CONFIRMED e intentan enviarlo sin número de guía, el sistema rechaza la operación. El pedido permanece en CONFIRMED y no se genera OrderShipped.

13. Ejemplos de transiciones prohibidas

Operación	¿Porque se rechaza?
Confirmar un pedido dos veces	El pedido ya no está en DRAFT.
Enviar un pedido sin número de guía	No cumple los requisitos de envío.
Cancelar un pedido enviado	La cancelación solo está permitida desde CONFIRMED.
Entregar un pedido sin enviarlo	El pedido debe estar en SHIPPED.
Confirmar un pedido sin artículos	No cumple las condiciones de confirmación.

Cuando una operación se rechaza, el estado del pedido no cambia ni se registra el evento correspondiente.

14. Reflexión breve: tres errores encontrados, cómo los solucionaste y qué aprendiste.

Durante el desarrollo de esta actividad tuvimos algunas dificultades al preparar el programa. Una de ellas fue instalar Python desde CMD, ya que al principio no sabíamos dónde se había instalado ni cómo ejecutarlo correctamente. También tuvimos problemas con el entorno virtual `.venv`, por lo que fue necesario revisar las rutas y los comandos para poder trabajar con el proyecto.

Otro de los retos fue realizar las pruebas y comprobar que el programa respondiera correctamente. Para ello seguimos los ejemplos de la guía, como el caso del pedido O013, donde se intenta confirmar dos veces y la segunda operación debe ser rechazada. Esto nos ayudó a entender mejor cómo funcionan las excepciones y por qué es importante comprobar tanto las operaciones exitosas como las que no están permitidas. En general, aprendimos más sobre la configuración de Python, el uso de la terminal y la importancia de realizar pruebas antes de dar por terminado un programa.

15. Errores y dificultades

Error 1 · Métodos de transición colocados fuera de la clase Order (AttributeError)

Aspecto	Descripción
Qué sucedió	Al ejecutar order.confirm() en la consola de Python, el sistema lanzó un error AttributeError: 'Order' object has no attribute 'confirm'.
Cómo se detectó y causa	Python no encontraba los métodos confirm(), ship(), cancel(), deliver() ni return_order() en la instancia de la entidad. La causa fue un problema de indentación y ubicación en app/domain.py: la clase DomainEvent se definió a mitad del archivo interrumpiendo la clase Order, y los métodos de transición quedaron pegados al margen izquierdo sin la sangría (tabulación) adecuada, convirtiéndose en funciones sueltas del módulo.
Solución	Se reestructuró app/domain.py moviendo la definición de DomainEvent arriba de la clase Order e indentando correctamente todos los métodos (confirm, ship, cancel, deliver, return_order) dentro del cuerpo de Order. Tras reiniciar la consola interactiva de Python, la invocación de order.confirm() funcionó correctamente.
Aprendizaje	En Python, la sangría define la pertenencia a un bloque de código o clase. Los métodos de una entidad deben estar correctamente alineados dentro de su declaración para formar parte de su interfaz pública y encapsular el comportamiento.

Error 2 · Rutas relativas en `cargar_csv.py` causaban `KeyError` y respuestas HTTP `404` en `pytest`

Aspecto	Descripción
Qué sucedió	La ejecución inicial de python -m pytest -q resultó en varios fallos (FAILED), arrojando errores KeyError: '0001' en las pruebas de dominio y 404 Not Found en las pruebas de la API REST.
Cómo se detectó y causa	El reporte de error de pytest mostró que load_fixture() no estaba encontrando ni cargando los datos de la carpeta datos/. La causa fue el uso de rutas relativas simples (como "datos/pedidos.csv"). Al ejecutar pytest desde la raíz o una subcarpeta, el directorio de trabajo actual (CWD) difería de la ubicación de cargar_csv.py, intentando leer archivos inexistentes y dejando el diccionario de pedidos vacío.
Solución	Se modificó cargar_csv.py para construir rutas absolutas dinámicas utilizando os.path.dirname(os.path.abspath(__file__)) concatenado con os.path.join(BASE_DIR, "datos", filename). Esto garantizó que load_fixture() siempre encuentre los CSV sin importar desde dónde se invoquen las pruebas. Resultado posterior: 21 passed.
Aprendizaje	Nunca se deben depender de rutas relativas simples para la carga de recursos o archivos de datos en proyectos de Python. Las rutas deben calcularse dinámicamente respecto a la ubicación del archivo ejecutable o módulo base.

Error 3 · Estructura de `NotSpec` y confusión de operandos en el operador unario

Aspecto	Descripción
Qué sucedió	Al intentar implementar el operador de inversión ~ (__invert__), se asumió inicialmente que NotSpec requería dos parámetros (left y right) al igual que AndSpec y OrSpec.
Cómo se detectó y causa	Al evaluar una especificación invertida como ~ActivePromotionSpec(), Python lanzó un error TypeError al instanciar NotSpec. Al revisar la firma del método __invert__(self) en la clase base Specification, se observó que la llamada era NotSpec(self), enviando un único argumento. A diferencia de AND y OR (operadores binarios), NOT es un operador unario.
Solución	Se ajustó la clase NotSpec para que su constructor (__init__) reciba únicamente un argumento (spec), y en su método is_satisfied_by(order) aplique la negación lógica sobre la única especificación guardada: return not self.spec.is_satisfied_by(order).
Aprendizaje	Los operadores lógicos binarios (&, `

16. Preguntas para el reporte

Pregunta 1. ¿Por qué se usa Decimal en lugar de float para precios?

Se utiliza Decimal porque permite realizar cálculos con cantidades de dinero de forma más precisa. Con float pueden aparecer pequeñas diferencias al trabajar con decimales, lo que podría afectar los totales y descuentos. En TechStore se utiliza Decimal para manejar los precios y evitar errores al calcular los importes de los pedidos.

Pregunta 2. ¿Cuál es la diferencia entre contar renglones y sumar unidades?

Contar renglones significa contar los productos registrados en el pedido, mientras que sumar unidades es obtener la cantidad total de artículos. Por ejemplo, el pedido O001 contiene dos renglones con 2 unidades cada uno, por lo que tiene 4 unidades en total. Esta diferencia es importante porque

TechStore establece un máximo de 20 unidades por pedido, independientemente del número de renglones.

Pregunta 3. ¿Qué ventaja tiene encapsular una regla en Specification?

Permite tener las reglas de negocio organizadas y utilizarlas en diferentes partes del programa sin repetir las mismas condiciones. En TechStore se aplica para comprobar los requisitos de los descuentos, como que el cliente sea VIP, que el pedido supere los $500 y que no tenga una promoción activa. Además, facilita modificar alguna condición sin tener que cambiar varias partes del código.

Pregunta 4. ¿Por qué los eventos se registran solo después de una transición válida?

Porque los Domain Events indican que una acción realmente se realizó. Por ejemplo, si se intenta enviar un pedido sin número de guía, el sistema rechaza la operación y no debe registrar OrderShipped. Por eso primero se comprueban las condiciones, después se actualiza el estado y finalmente se genera el evento. Así se evita registrar acciones que no ocurrieron.

Pregunta 5. ¿Qué parte del código podrías reutilizar si reemplazaras FastAPI por otra interfaz?

Se podría conservar la lógica de negocio que se encuentra en `app/domain.py` y las reglas de descuentos de `app/specifications.py`, ya que no dependen directamente de FastAPI. Si el programa se utilizara desde una aplicación de escritorio o desde la consola, estas clases y sus validaciones seguirían funcionando. Lo que tendría que modificarse sería la forma de recibir las solicitudes y mostrar las respuestas al usuario.

LINK GITHUB : https://github.com/Mordekay7u7/TechStore_U4